

Library Management System Documentation

1. Project Overview

The Library Management System is a Java-based desktop application implementing multiple design patterns to create a robust, maintainable, and extensible library management solution. The system handles book management, user authentication, loan tracking, and administrative functions through a graphical user interface.

2. Design Patterns Implementation by Package

2.1. library.management.system.singleton

Pattern: Singleton

- **Key Class:** `DatabaseConnection.java`
- **Implementation:**
 - Ensures single database connection instance
 - Thread-safe implementation using double-checked locking
 - Handles connection recreation if closed
 - Properties:

```
private static DatabaseConnection instance;  
private Connection connection;  
private final String url = "jdbc:mysql://localhost:3306/LibraryManagementSystem";
```

2.2. library.management.system.command

Pattern: Command

- **Key Classes:**
 - `Command.java` (interface)
 - `AddBookCommand.java`
 - `DeleteBookCommand.java`

- Invoker.java
- BookDAO.java

- **Implementation:**

- Encapsulates book operations as commands
- Provides undo capability
- Separates command requester from command executor
- Example:

```
public class Invoker {
    private Command command;
    public void setCommand(Command command) {
        this.command = command;
    }
    public void executeCommand() {
        command.execute();
    }
}
```

2.3. library.management.system.strategy

Pattern: Strategy

- **Key Classes:**

- UserActionStrategy.java (interface)
- LoginStrategy.java
- RegisterStrategy.java
- UserActionContext.java

- **Implementation:**

- Defines family of authentication algorithms
- Makes authentication strategies interchangeable
- Separates algorithm implementation from usage
- Example:

```
public interface UserActionStrategy {
    void execute(String userName, String email, String password, JFrame frame);
}
```

2.4. library.management.system.factory

Pattern: Factory

- **Key Classes:**

- BookDisplay.java (interface)
- AllLoanedBooks.java
- BookDisplayFactory.java

- **Implementation:**

- Creates book display objects without exposing creation logic
- Provides common interface for book display
- Example:

```
public class BookDisplayFactory {  
    public static BookDisplay getBookDisplay() {  
        return new AllLoanedBooks();  
    }  
}
```

2.5. library.management.system.operationState

Pattern: State

- **Key Classes:**

- OperationState.java (interface)
- AddLoanRecordState.java
- UpdateBookCopiesState.java
- OperationContext.java

- **Implementation:**

- Manages different states of book operations
- Allows object to alter behavior when internal state changes
- Example:

```
public class OperationContext {  
    private OperationState currentState;  
    public void setState(OperationState state) {  
        this.currentState = state;  
    }  
    public void execute() {  
        if (currentState != null) {  
            currentState.execute();  
        }  
    }  
}
```

3. Database Schema and Operations

3.1. Tables

1. Books Table

```
CREATE TABLE books (  
    book_id INT PRIMARY KEY AUTO_INCREMENT,  
    book_name VARCHAR(255),  
    author VARCHAR(255),  
    copies_count INT  
);
```

2. Users Table

```
CREATE TABLE user (  
    email VARCHAR(255) PRIMARY KEY,  
    username VARCHAR(255),  
    password VARCHAR(255)  
);
```

3. LoanedBooks Table

```
CREATE TABLE LoanedBooks (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    book_id INT,  
    book_name VARCHAR(255),  
    borrower_name VARCHAR(255),  
    borrower_email VARCHAR(255),  
);
```

4. System Workflow

4.1. User Authentication Flow

1. User selects login/register action
2. Strategy pattern determines authentication method
3. UserActionContext executes appropriate strategy
4. On success:
 - Admin users directed to AdminPage

- Regular users directed to UserPage

4.2. Book Management Flow

1. Admin initiates book operation
2. Command pattern handles operation
3. BookDAO executes database operation
4. State pattern manages operation state
5. Factory pattern creates appropriate display

5. Security Features

5.1. Database Security

- Connection pooling through Singleton pattern
- Prepared statements prevent SQL injection
- Secure password handling
- Connection timeout handling

5.2. User Authentication

- Email uniqueness verification
- Password validation
- Role-based access control
- Session management

6. Error Handling

6.1. Database Errors

- Connection failure handling
- Query execution error management
- Resource cleanup
- Transaction rollback

6.2. User Input Validation

- Email format verification

- Required field validation
- Input sanitization
- Error message display

7. System Requirements

7.1. Technical Requirements

- Java Runtime Environment 8+
- MySQL Server 5.7+
- Minimum 4GB RAM
- 100MB free disk space

7.2. Dependencies

- JDBC MySQL Connector
- Swing GUI library
- Java Collections Framework

8. Future Enhancements

1. Book reservation system
2. Fine calculation module
3. Email notification system
4. Report generation
5. Book category management

Library Management System - Android Firebase Implementation

1. Project Overview

The Library Management System is an Android application that leverages Firebase services for authentication and data management. The system provides distinct interfaces for users and librarians, focusing on book management and borrowing functionality.

2. System Architecture

2.1. Firebase Services Integration

1. Firebase Authentication

- Email/password authentication
- User role management (Librarian/User)
- Secure login/registration process

2. Firebase Firestore

- Book collection management
- Loan records
- User profiles
- Real-time data synchronization

3. Firebase Storage

- Book PDF file storage
- Secure file access control
- Download functionality

2.2. Database Collections Structure

```
firestore/  
├─ books/  
│   └─ bookId/  
│       │   └─ title: string  
│       │   └─ author: string  
│       │   └─ available: boolean  
│       │   └─ pdfUrl: string  
│       └─ timestamp: timestamp  
├─ users/  
│   └─ userId/  
│       │   └─ email: string  
│       │   └─ name: string  
│       │   └─ role: string  
│       └─ timestamp: timestamp  
└─ loans/  
    └─ loanId/  
        │   └─ bookId: string  
        │   └─ userId: string  
        │   └─ borrowDate: timestamp  
        └─ status: string
```

3. Design Patterns Implementation

3.1. Command Pattern

- Used for book management operations
- Implemented in librarian operations:
 - Add book command
 - Remove book command
 - Update book command

3.2. Strategy Pattern

- Handles different user authentication approaches
- Manages different user roles (Librarian/User)
- Controls access permissions

3.3. Singleton Pattern

- Firebase instance management
- Application configuration
- Session management

3.4. Factory Pattern

- Creates book display views
- Manages UI component creation
- Handles different user interfaces based on role

3.5. Adapter Pattern

- Adapts Firebase data for RecyclerView display
- Handles book list display
- Manages loan record display

Project Link on Github

[Library Management System APP & Desktop Project](#)

Team Members

Name	ID
Ashraf Gamal Rafeek	2100522
Beshoy Emad Fawzy	2100579
Asmaa Mohamed Abdel Wakeel	2100527
Beshoy Usama Zakaria	2100578
Asmaa Atef Hassan	2100528
Dina Sadiq Badr	2100601

Thanks for Reading